

## **Project Overview**

**Project Name:** Nómada

### **Description:**

Nómada is a web platform designed to recommend tourist destinations in Costa Rica. It allows users to explore destinations by category, read and post experiences, and interact with an AI assistant for personalized recommendations.

## **2. Target Users**

**Visitor:** Any person accessing the platform without an account.

**Registered User:** A user who has created an account to interact with the platform.

**System Owner:** The creator of the website (only one) who manages all content.

## **3. User Stories**

### **3.1. Visitor (No Account)**

**US-01:** As a Visitor, I want to view destinations by category so that I can explore places of interest.

**US-02:** As a Visitor, I want to see destination details (photos, description, cost) so that I can decide where to go.

**US-03:** As a Visitor, I want to read user comments and ratings so that I can learn from others' experiences.

**US-04:** As a Visitor, I want to use the AI chat assistant so that I can get personalized travel recommendations.

**US-05:** As a Visitor, I want to register a new account so that I can become a registered user.

### **3.2. Registered User**

**US-06:** As a Registered User, I want to log into my account so that I can access my profile.

**US-07:** As a Registered User, I want to post comments on destinations so that I can share my travel experiences.

**US-08:** As a Registered User, I want to rate destinations (1-5 stars) so that I can help other travelers.

### 3.3. System Owner (Creator of the Website)

**US-09:** As the System Owner, I want to add new destinations so that the platform stays updated.

**US-10:** As the System Owner, I want to edit destination information so that I can correct or improve content.

**US-11:** As the System Owner, I want to delete inappropriate comments so that the community remains respectful.

**US-12:** As the System Owner, I want to view all registered users so that I can monitor platform activity.

### 4. Non-Functional Requirements

**NFR-01 (Performance):** The platform must load in less than 3 seconds.

**NFR-02 (Responsive Design):** The interface must work on mobile, tablet, and desktop.

**NFR-03 (Security):** User passwords must be encrypted (hashing).

**NFR-04 (Database):** The system must use SQL (MySQL) with referential integrity.

**NFR-05 (OOP Paradigm):** The code must follow Object-Oriented Programming principles.

**NFR-06 (Version Control):** All code must be hosted on GitHub with descriptive commits.

**NFR-07 (Project Management):** Tasks must be tracked in Azure Boards (Epics, Tasks, States).

**NFR-08 (AI Integration):** The system must integrate an AI API (OpenAI/Gemini) for chat.

**NFR-09 (Documentation):** All documentation and comments must be in English.

## 5. System Modules

### Module 1: User Management

User registration

User login/logout

**Two roles:** Visitor, Registered User

System Owner manages everything manually (not through a role in the system)

### Module 2: Destination Catalog

View destinations by category (Beaches, Parks, Adventure, Hot Springs)

View destination details (photos, description, location, daily cost)

System Owner adds, edits and deletes destinations (manually)

### **Module 3: Experiences & Comments**

Registered users can post comments  
Registered users can rate destinations  
All users can read comments  
System Owner can delete comments

### **Module 4: AI Assistant**

Chat interface for all users  
Integration with OpenAI or Gemini API  
Provides travel recommendations based on user input

## **6. Data Model**

### **Users Table**

id (integer, primary key)  
name (string)  
email (string, unique)  
password (string, hashed)  
role (string: visitor, registered)  
created\_at (datetime)

### **Destinations Table**

id (integer, primary key)  
name (string)  
description (text)  
location (string)  
category (string: beaches, parks, adventure, hot springs)  
daily\_cost (decimal)  
main\_photo (string)  
photo2 (string)  
photo3 (string)  
created\_at (datetime)

## Comments Table

id (integer, primary key)  
user\_id (integer, foreign key to users)  
destination\_id (integer, foreign key to destinations)  
text (text)  
rating (integer, 1-5)  
created\_at (datetime)

## Relationships:

One user can have many comments  
One destination can have many comment

## 7. Technical Stack

**Backend:** Python with Flask  
**Frontend:** HTML, CSS, JavaScript  
**Database:** MySQL  
**ORM:** SQLAlchemy (optional)  
**AI API:** OpenAI API or Gemini API  
**Version Control:** GitHub  
**Deployment:** Local execution (optional cloud hosting)

## 8. Glossary

**Visitor:** A user who has not logged in.  
**Registered User:** A user with an account.  
**System Owner:** The creator of the website (only one person) who manages destinations and comments.  
**Destination:** A tourist place in Costa Rica.  
**Experience:** A comment or review left by a user.  
**AI Assistant:** Chatbot that recommends destinations.